

BINV2150-A – Javascript : avancé

Séance 3 – Middlewares avec Express

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École
Léonard
de Vinci

Rappels web1

Serveur REST



Un serveur REST est un serveur qui respecte entre autre les contraintes suivantes :

- l'utilisation des URL pour accéder aux ressources,
- l'utilisation des méthodes HTTP (GET, POST, PUT, DELETE) pour effectuer des opérations sur les ressources,
- l'utilisation des réponses du serveur au format standard (par exemple : JSON) ;
- le serveur ne garde pas d'état (de mémoire des requêtes précédentes).

Rappels web1

Serveur REST

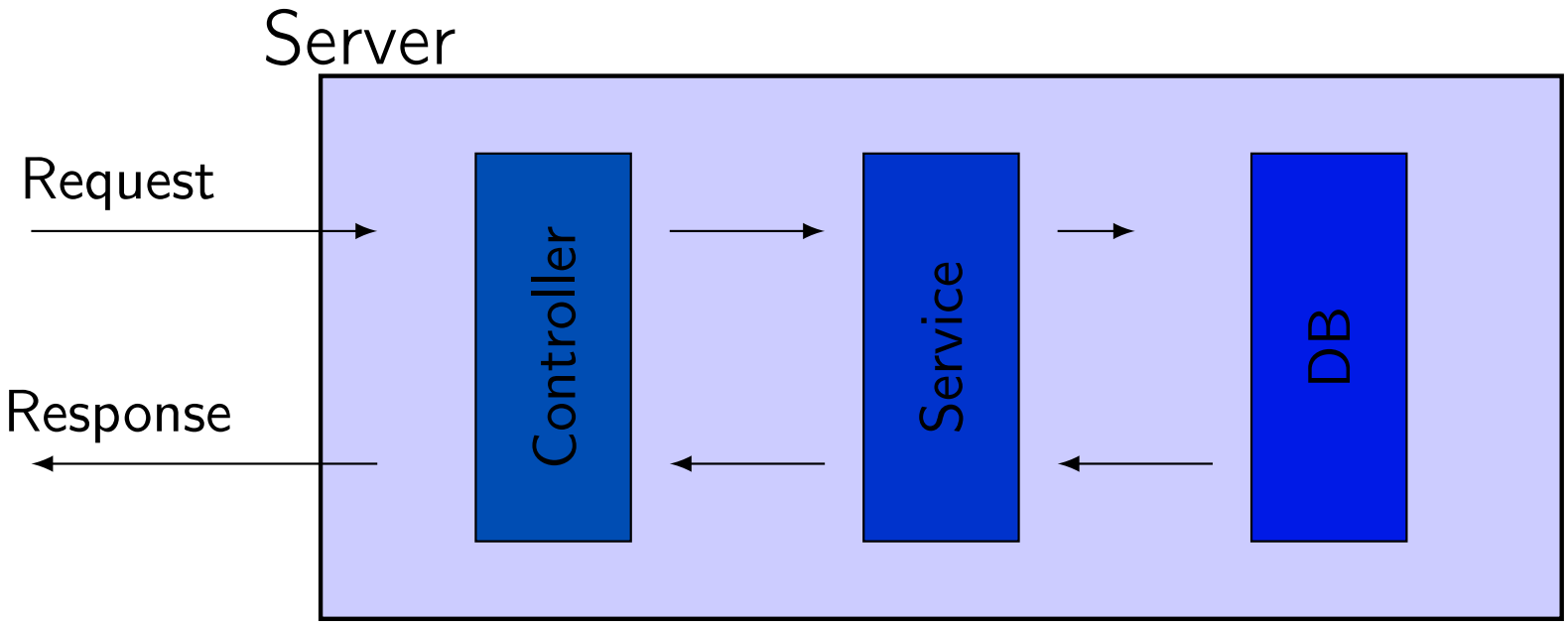


Figure – Architecture d'un serveur REST

Express

Flow

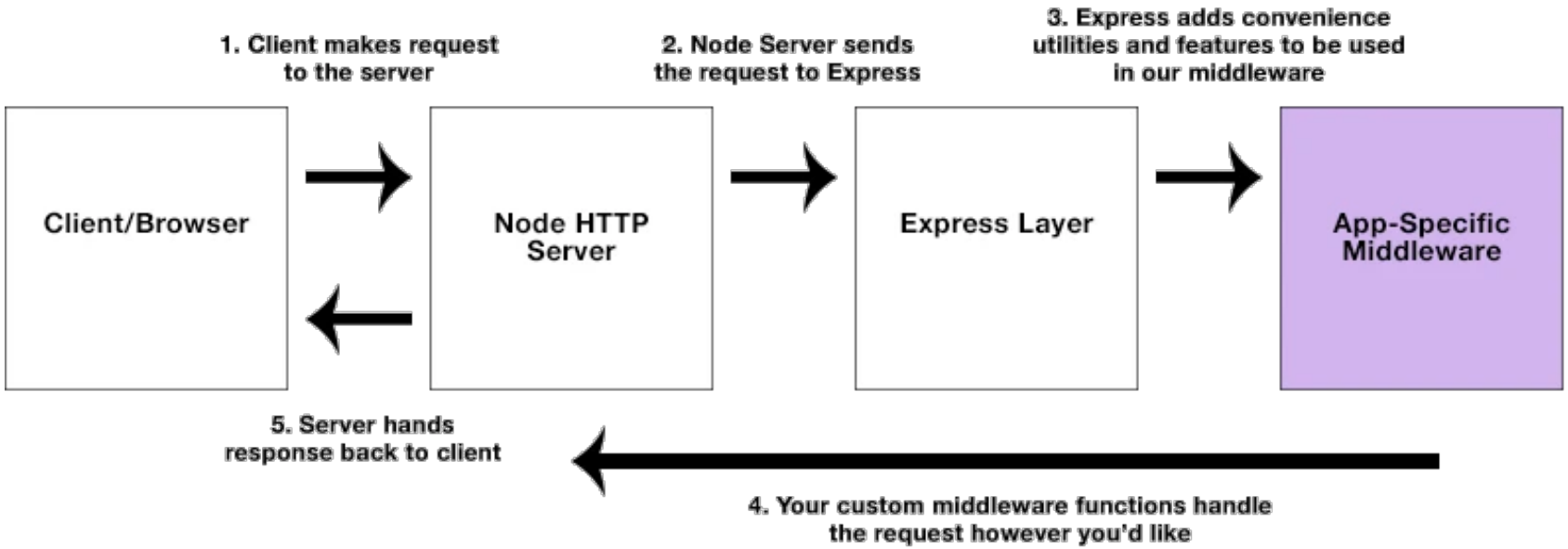


Figure – Flux d'une requête vers une application Express, source : turing.edu

Express

Middleware

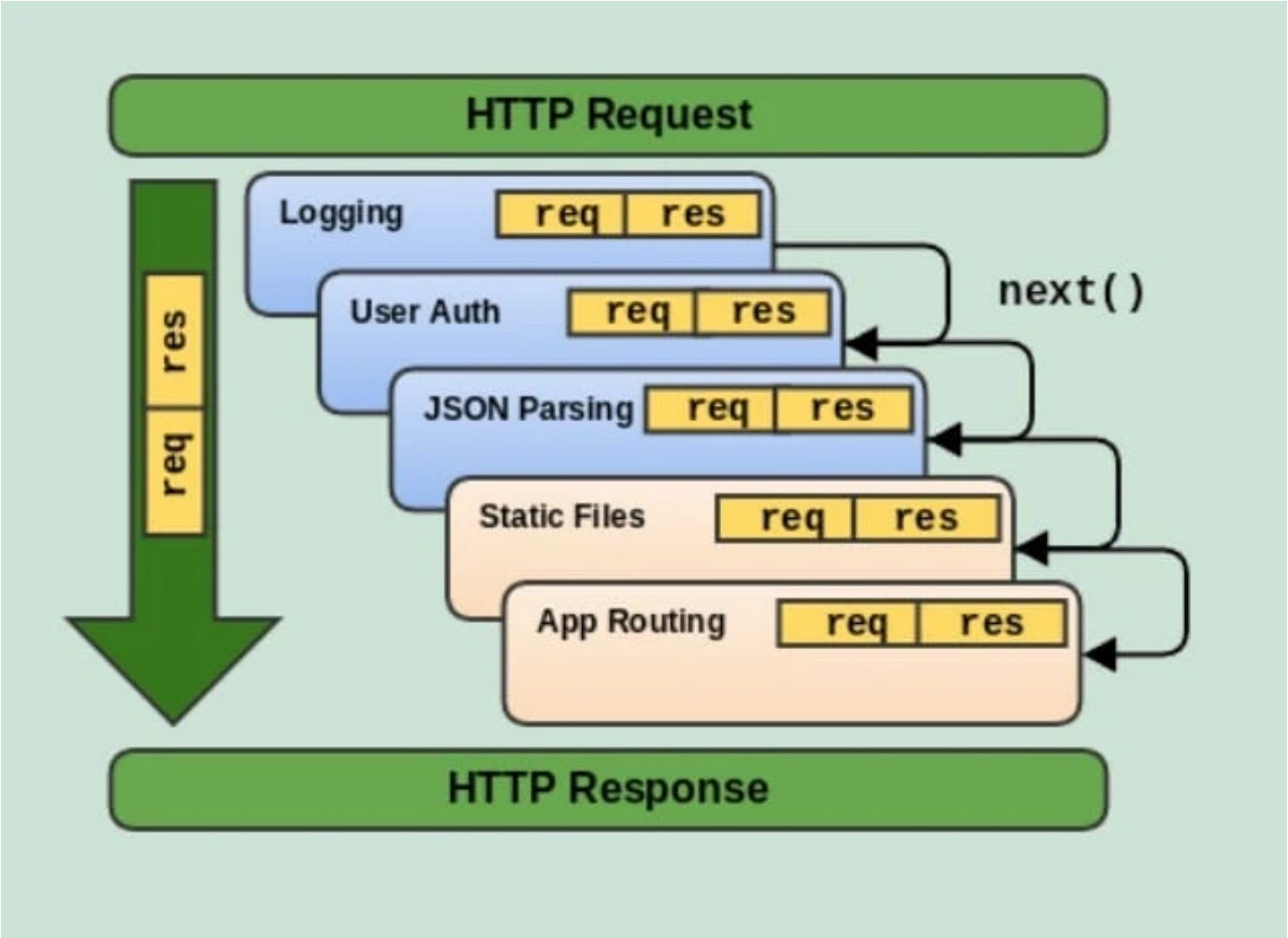


Figure – Les fonctions Middleware et Express, source : dev.to



1. Les fonctions middleware

Middleware

Express Router

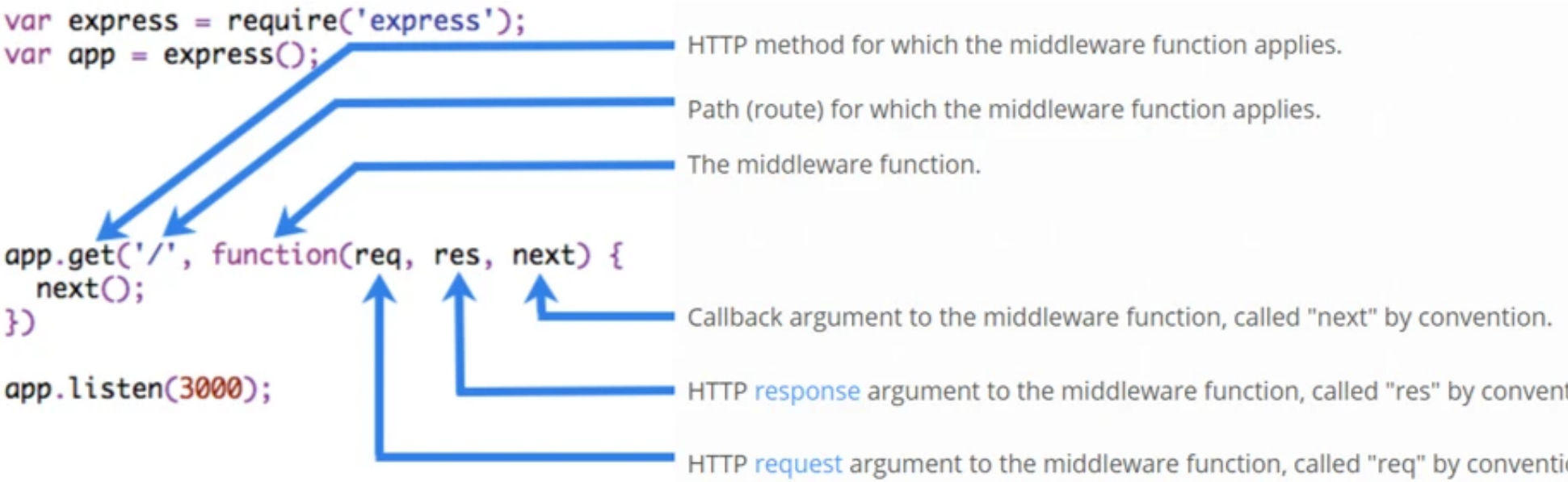


Figure – Une fonction middleware

Qu'est-ce qu'un middleware ?



- **Fonction** exécutée entre la requête (client) et la réponse (serveur).
- Peut **lire/modifier** `req`, `res`, ou décider de **terminer** la réponse.
- Appelle `next()` pour passer au suivant (ou transmet une erreur à `next(err)`).
- Trois niveaux : **app** (`app.use`), **router** (`router.use`), **route**.
- L'**ordre d'enregistrement** détermine l'ordre d'exécution.
- Un middleware peut **court-circuiter** la chaîne (`res.end()`).

Middleware au niveau application

Logger simple



```
const app = express();

function logger(req: Request, res: Response, next: NextFunction) {
  console.log(`${req.method} ${req.url}`);
  next(); // passe au middleware suivant
}

app.use(logger);

app.get("/", (req, res) =>{
  res.send("Hello world!");
});
```

Middlewares au niveau application

Middlewares intégrés à Express



- `express.json()` : parse le corps en JSON.
- `express.urlencoded()` : parse données de formulaires.
- `express.static()` : sert des fichiers statiques.

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use("/public", express.static("public"));
```

Middleware au niveau router

Logger spécifique



```
1  const adminRouter = express.Router();
2
3  adminRouter.use((req, res, next) =>{
4    console.log(`Accès àla zone admin: ${req.method} ${req.url}`);
5    next();
6  });
7
8  adminRouter.get("/", (req, res) =>{
9    res.send("Liste des administrateurs");
10 });
11
12 export default adminRouter;
```

Middleware au niveau route

Authentification simple



```
function auth(req: Request, res: Response, next: NextFunction) {
  if (req.headers.authorization !== "secret-token") {
    return res.status(401).send("Unauthorized");
  }
  next();
}

app.get("/admin", auth, (req, res) =>{
  res.send("Bienvenue admin!");
});
```

Middleware d'erreur

Gestion des erreurs



```
function errorHandler(err: Error, req: Request, res: Response, next:
    NextFunction) {
  console.error(err.stack);
  res.status(500).send("Something broke!");
}

app.use(errorHandler);
```

Résumé



- Middleware = brique de traitement entre `req` et `res`.
- Trois niveaux : application, router (controller), route.
- Peut logger, parser, sécuriser, transformer.
- L'ordre d'exécution est essentiel.